Name : Vansh Sharma

Roll No : 2401109

Batch : T22

Subject : ASD&D

# Experiment-03

**Aim** : To deploy multi-service applications under Docker Compose.

**This experiment maps to LO4.**

**LO4** : Containerize, orchestrate and deploy applications using Docker, Docker Compose, Kubernetes, and Cloud Infrastructure tools.

**Theory :**

Docker Compose is a tool used for defining, configuring, and running **multi-container Docker applications**. It allows developers to manage multiple services of an application using a single configuration file called compose.yaml or docker-compose.yml.

In a multi-service application, different components such as a web server, application server, database, and cache run in separate containers. Docker Compose simplifies the process of creating, networking, and managing these containers together.

A Compose file is written in **YAML format** and contains the details of each service, including:

- Docker images to use
- Ports to expose
- Environment variables
- Dependencies between services
- Networks and storage configurations

In this experiment, a multi-service application is deployed using four services:

1. **Nginx Web Server** ○ Acts as the web service and handles HTTP requests. ○

   Provides communication between users and the application.

2. **Flask Application**

- Acts as the application layer.
- Runs the Python-based web application inside a Docker container.

3. **MySQL Database** ○ Stores and manages application data.
   - Runs as an independent database container.
4. **Redis Cache** ○ Provides fast in-memory data storage.
   - Improves application performance by caching frequently accessed data.

Docker Compose creates a separate container for each service and connects them through a default Docker network, allowing the containers to communicate with each other. The command: docker compose up -d --build builds required images, creates containers, establishes networking, and starts all services in the
background. The command:

docker compose down stops and removes the containers and network

created by Docker Compose.

**Advantages of Docker Compose:**

- Simplifies deployment of multi-container applications.
- Provides consistent application environments.
- Allows easy scaling of services.
- Reduces manual container management.
- Enables communication between related services through Docker networking.

Compose.yaml :

```
services:   web:
  image: nginx:latest
```

ports: -

"8081:80"

depends_on:

- app

app: build:

./app ports: -

"5000:5000"

depends_on:

- database

- cache database: image: mysql:8 environment: MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: testdb cache:

image: redis:latest

| Command | Purpose |
|---|---|
| docker --version | Checks the installed Docker version to verify Docker is available. |
| docker compose version | Checks the installed Docker Compose version. |
| mkdir dockercomposedemo | Creates a new project folder named docker-compose-demo. |

| Command | Purpose |
|---|---|
| cd dockercomposedemo | Moves into the project folder where Docker Compose files are stored. |
| dir | Displays the files and folders inside the current directory. |

| | |
|---|---|
| notepad app.py | Creates/opens the app.py file using Notepad to write the Flask application code. |
| type app.py | Displays the contents of the app.py file in Command Prompt. |
| docker compose | Creates and starts the services defined in compose.yaml in detached mode (runs in the background). |
| docker compose --build | Builds Docker images again and starts all services in the background. Used when application code or Dockerfile changes. |
| docker ps | Shows currently running Docker containers. |
| docker ps -a | Shows all containers, including stopped and exited containers. |
| docker compose down | Stops and removes the containers, network, and resources created by Docker Compose. |
| docker compose | Displays logs from all services running through Docker Compose. |
| docker compose | Shows the status of services managed by Docker Compose. |
| docker info | Displays detailed information about the Docker installation and Docker Engine status. |
| docker version | Displays Docker client and server version information. |

**Output :**



**Conclusion :**

From this experiment, I understood that Docker Compose is used to deploy and manage **multi-service applications** by running multiple containers together. I learned how to create a compose.yaml file to define different services and how Docker Compose automatically creates, connects, and manages these containers.

I successfully deployed a 4-service application consisting of **Nginx, Flask application, MySQL database, and Redis cache**. I understood that each service runs independently in its own container but can communicate with other services through Docker networking.

I also learned how to build, start, check, and stop services using Docker Compose commands such as docker compose up, docker ps, and docker compose down. This experiment helped me understand how Docker Compose simplifies the deployment of complex applications with multiple components.